

Proyecto de *Spark* para Comparar Arquitecturas de Ejecución en Tiempo Real

1. Motivación

En Ciencia de Datos es vital el procesamiento de información masiva en *batch* para entrenar modelos de *machine learning*. Normalmente, la arquitectura para el entrenamiento por lotes se da por sentado. Sin embargo, para el procesamiento de datos en tiempo real hay varios factores a considerar:

- Tipos de fuentes de datos y velocidad del flujo
- Ventanas de tiempo y marcas de agua
- Como guardar los resultados
- Llegada de datos (exactamente una vez, más de una vez o al menos una vez)

En consecuencia, es importante dominar aspectos relacionados con la arquitectura de la plataforma donde se ejecutan aplicaciones de flujos en tiempo real.

La motivación de este proyecto es desarrollar una aplicación usando *Spark Structure Streaming* y *Kafka* para capturar datos en tiempo real. Una vez con suficiente información recuperada del flujo de entrada, entrenar un modelo de supervisado (clasificación o regresión) y hacer predicciones del modelo con datos que continúan llegando en tiempo real. Es decir, se debe hacer la captura de datos en tiempo real para calcular algunos estadísticos o aplicar algunas técnicas de aprendizaje no supervisado (*clustering*). A continuación guardar los datos en disco para montar un modelo supervisado simple. Es posible, en lugar de estadísticos, usar técnicas de aprendizaje no supervisado *online* como: STREAM-LOF, OLOF, DBSCAN, ...

Dado que cada equipo es de dos personas, deben realizar comparaciones de la ejecución con sus computadoras personales con o sin GPUs (*spark* local), *Google Colab* o AWS para constatar, con la misma aplicación, el desempeño de las diferentes plataformas. Deben usar *Spark* UI que se puede acceder en la instalación local mediante:

<http://localhost:4040>

En caso de usar AWS o *Google Colab*, los parámetros de ejecución de *Spark Applications* UI se descargan de un servidor remoto y debe establecer un túnel para acceder a los datos de la ejecución. Deben usar al menos **dos** (2) métricas de las indicadas a continuación para comparar el desempeño de sus arquitecturas:

- Tiempo de ejecución de los *jobs* en ambas arquitecturas
- Tiempos de los *shuffle* entre cada *stage* ya que son operaciones lentas. Por ejemplo, *join* y *reduceByKey* hacen *shuffle*.
- Cantidad de operaciones de I/O en cada *stage*, ya que al igual que los *shuffles*, son operaciones lentas.
- Están *Scheduler Delay* (Retraso esperando recursos), *Executor Run Time* (Tiempo real de computación) y *GC Time* (Tiempo de recolección de basura) que deben revisar para cada arquitectura
- *Spill* (Memoria/Disk): Si los datos exceden la memoria disponible del *executor*, Spark "derrama" (*spills*) a disco. Un alto *spill* es un signo de rendimiento degradado y memoria RAM insuficiente.
- Tasa de *streaming* para visualizar el progreso de cada consulta como una serie temporal. Las métricas son: *input rate*, *processing rate*, *batch duration* y la cola de *batches* pendientes.
- *Environment* aunque no es una métrica de rendimiento directo, es fundamental para la comparación.

Deben dar los detalles de hardware de cada una de las dos arquitecturas que usarán. Lo ideal sería hacerlo en una tabla comparativa y emitir opiniones cualitativas y cuantitativas sobre el potencial de cada arquitectura.

2. Objetivo Específicos

- Desarrollar una aplicación en *pySpark* que capture datos en tiempo real usando *Spark Structure Streaming* y *Kafka*
- Probar que funcione localmente (con o sin GPUs), o en *Google Colab* o en *AWS*.
- Calcular indicadores estadísticos clásicos, o modelos no supervisados en tiempo real, sobre el tráfico en tiempo real.
- Tomar los datos de un *streaming* en tiempo real y si no es posible con un *socket* simulando ventanas de tiempo para poder tener estimaciones de tiempo acopladas a la arquitectura seleccionada
- Una vez que tengan suficientes datos guardados de la captura en tiempo real, entrenar un modelo supervisado de *machine learning*
- Reactivar el flujo en tiempo real y ahora clasificar los nuevos casos que vienen en la segunda tanda de *streaming*.
- Comparar la ejecución de la misma aplicación, con los mismos datos, en las diferentes plataformas que seleccionaron usando *Spark Application UI*. No olviden colocar capturas de pantalla que justifiquen sus argumentos.

3. Funcionalidades

El flujo de datos constante puede provenir de una infraestructura:

- IoT
- Casco de electroencefalogramas
- Sensores de robots
- Bolsa de valores,
- Datos financieros: criptomonedas, precios de activos como oro o petróleo,
- Tráfico sobre algún servidor de red
- Datos de partidos de futbol
- Estimaciones de transporte público: buses, bicicletas, ...
- Datos meteorológicos, etc.

Por ejemplo, los cascos de electroencefalogramas que tiene el ITAM poseen 16 sensores y cada uno lee 256 lecturas por segundo. Esto hace un total de 4096 lecturas por segundo. El flujo que seleccionen debe manejar un flujo, al menos, de esa magnitud.

Para las estadísticas básicas del *streaming* en tiempo real deben considerar al menos:

- Los valores, en caso de ser continuos, (por ejemplo, un sensor de temperatura), deben definirse por rangos y reportar las estadísticas asociadas a esos rangos.
- Determinar los mínimos y máximos de los valores del flujo(s) de entrada y calcular, al menos, los promedios y varianza. La cadencia de emisión de estos reportes debe ser muy frecuente porque les servirá para construir una visualización dinámica. No olviden considerar un modelo de aprendizaje no supervisado online si les resulta útil.
- Para la visualización animada de los indicadores estadísticos para el flujo de entrada pueden usar PowerBI, Tableau o Qlik Sense. Además, deben mostrar los tiempos de procesamiento en las diferentes configuraciones que utilicen: (*Spark local* con o sin GPUs), *Google Colab* y/o AWS.

4. Informe

El informe debe explicar las dos arquitecturas que utilizarán, la estructura de los datos, estadísticos calculados, colocar las diferencias importantes que notará durante la ejecución de la aplicación en los dos entornos de ejecución que utilicen. Deben tener unas conclusiones que interpreten los resultados.

Como parte de la evaluación, deben mostrar la ejecución de la aplicación al profesor de la asignatura antes de la última semana del semestre y se les pedirá una pequeña modificación, como parte de la evaluación del examen final del curso.

Finalmente, la fecha tope para entregar su informe y código a través de Canvas, el viernes de la 16^{ava} semana.